Connections Guide

How to set up each integration for the NovaCRM onboarding build. Companion file: `IMPLEMENTATION_PLAN.md`.

Items marked **Verify** come from memory or third-party summaries, not from a doc I opened today. Check them against the linked official docs before relying on them.

---

0. Order of setup

Do these in this order. The first two are where people lose hours.

1. Retell AI: make one real call to your phone.
2. Rocketlane: confirm the trial has API keys and automations.
3. Tunnel (ngrok).
4. Google Cloud + Gmail Pub/Sub.
5. Slack.
6. LLM API key.
7. PostgreSQL (Docker).

---

1. Environment variables

```
# Core
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/app
MIG_SOURCE_URL=postgresql://postgres:postgres@localhost:5432/mig_source
MIG_DEST_URL=postgresql://postgres:postgres@localhost:5432/mig_dest
PUBLIC_BASE_URL=https://<your-subdomain>.ngrok-free.app
PROVIDER_MODE=real            # real | fake

# LLM
LLM_API_KEY=
LLM_MODEL=

# Gmail
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
GOOGLE_REFRESH_TOKEN=
GMAIL_PUBSUB_TOPIC=projects/<project-id>/topics/gmail-deals
GMAIL_PUSH_TOKEN=             # random string you generate
CS_INBOX_ADDRESS=

# Retell
```

```
RETELL_API_KEY=
RETELL_AGENT_ID=
RETELL_FROM_NUMBER=           # E.164, e.g. +14155550123

# Rocketlane
ROCKETLANE_API_KEY=
ROCKETLANE_TEMPLATE_ENTERPRISE=
ROCKETLANE_TEMPLATE_GROWTH=
ROCKETLANE_OWNER_EMAIL=
ROCKETLANE_CSM_ENTERPRISE=
ROCKETLANE_CSM_GROWTH=

# Slack
SLACK_BOT_TOKEN=xoxb-...
SLACK_INTERNAL_MEMBER_IDS=    # comma-separated user IDs to add to each channel
```

Commit `.env.example` only. Never commit `.env`.

---

2. Tunnel (ngrok)

Gmail Pub/Sub and Retell both need a public HTTPS URL.

```
ngrok http 8000
```

- Copy the HTTPS URL into `PUBLIC_BASE_URL`.
- The free URL changes on restart unless you claim the free static domain in the ngrok dashboard. Claim it, or you will be re-pasting URLs into Pub/Sub and Retell all day.
- Webhook URLs:
- Gmail: `{PUBLIC_BASE_URL}/webhooks/gmail?token={GMAIL_PUSH_TOKEN}`
- Retell: `{PUBLIC_BASE_URL}/webhooks/voice`

---

3. Retell AI (voice call)

Docs: https://docs.retellai.com

Why Retell: Vapi's free numbers are US inbound-only, so Vapi needs a second (Twilio) account to dial out. Retell-managed numbers can call international destinations directly.

Cost (from Retell's pricing page, checked today): $10 free credits on signup. Voice infrastructure $0.055/min. Telephony to India $0.15/min. A Retell number is about $2/month, billed to a card. LLM and voice are extra. Budget roughly $0.25 to $0.35 per minute to India (my estimate), so about 30 one-minute test calls on the free credits.

Setup

1. Sign up at retellai.com. Add a card if number purchase requires it. **Verify** whether the free credits cover the number.
2. **Phone Numbers > Buy number.** Pick a US number. Save it as `RETELL_FROM_NUMBER`.
3. **Check international calling.** Confirm India is enabled for outbound on your workspace (see "International Calling" in the docs).
4. **Create an agent** (single-prompt agent is enough). - Begin message: "Hi {{ae_name}}, this is the NovaCRM onboarding assistant calling about the {{customer_name}} deal." - Prompt, in short:
    - Ask one question: is the customer on the Enterprise plan or the Growth plan?
    - If the answer is clear, repeat it back and ask for a yes/no confirmation.
    - If the answer is hedged or unclear, ask once more. If still unclear, say a teammate will follow up and end the call.
    - Never suggest a tier. Never accept anything other than Enterprise or Growth.
    - Keep the call under 60 seconds.
    - Enable voicemail detection and set it to hang up.
5. **Post-call analysis.** Add custom fields: - `plan_tier`: enum `enterprise`, `growth`, `unclear` - `confirmed`: boolean (AE explicitly confirmed after the read-back)
6. **Webhook.** Set the agent's webhook URL to `{PUBLIC_BASE_URL}/webhooks/voice`.
7. **API key.** Settings > API Keys. Save as `RETELL_API_KEY`. Save the agent ID as `RETELL_AGENT_ID`.

Start a call

Verify the exact path and field names in the API reference ("Create Phone Call").

```
curl -X POST https://api.retellai.com/v2/create-phone-call \
  -H "Authorization: Bearer $RETELL_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "from_number": "+14155550123",
    "to_number": "+9198XXXXXXXX",
    "override_agent_id": "'$RETELL_AGENT_ID'",
    "metadata": {"run_id": "RUN_UUID", "attempt": 1},
    "retell_llm_dynamic_variables": {"ae_name": "Jai", "customer_name": "Acme Corp"}
  }'
```

The response includes a `call_id`. Store it in `call_attempts.provider_call_id`.

Webhook events

Verify event names and payload shape in the "Webhook" docs page.

Event	Use
`call_started`	Optional, update the dashboard
`call_ended`	Has `disconnection_reason` (no answer, voicemail, busy, hangup). Use it to detect no-answer
`call_analyzed`	Has the post-call analysis with your custom fields. Resume the graph here

Handler rules:

1. Verify the signature header (`x-retell-signature`) with your API key. The SDK has a verify helper. **Verify** the header name.
2. Read `run_id` from `call.metadata` .
3. Dedupe on `call:{call_id}:{event}` .
4. Return 200 immediately. Resume in a background task.
5. Map to a `CallResult` : - `plan_tier` in (`enterprise` , `growth`) and `confirmed=true` : confirmed - no answer, voicemail, busy, failed: `no_answer` - everything else: `unclear`

Fallback: Vapi + Twilio

Only if Retell cannot reach your number.

1. Twilio trial account. You get one number. Verify your Indian mobile under Verified Caller IDs. Enable India under Voice > Geo Permissions.
2. Vapi dashboard: Phone Numbers > Import > Twilio (Account SID + Auth Token).
3. `POST https://api.vapi.ai/call` with `assistantId` , `phoneNumberId` , `customer.number` , `metadata` .
4. Enable the `end-of-call-report` server message on your server URL.

Trial limits: calls only to verified numbers, a short Twilio trial message plays before the call, 10-minute cap. The trial message will be audible in your demo.

---

4. Rocketlane

Docs: https://developer.rocketlane.com

Account and API key

1. Start a free trial at rocketlane.com.
2. Profile icon > **Settings > API > Create API key**. Save as `ROCKETLANE_API_KEY` .
3. Base URL: `https://api.rocketlane.com/api/1.0`
4. Header on every request: `api-key: <key>`

Smoke test (**Verify** the path in the API reference):

```
curl https://api.rocketlane.com/api/1.0/projects \
  -H "api-key: $ROCKETLANE_API_KEY" -H "accept: application/json"
```

If this fails on the trial, email Karthik immediately. Everything depends on it.

Users

Create or invite three team members: a Project Owner, a Project Manager, and two CSMs (one "dedicated", one "pooled"). Using `you+pm@gmail.com` style aliases works if Rocketlane accepts them. Note each user's email or ID.

Templates (built in the UI)

Create two project templates, each with the 4 phases and about 15 tasks in total.

	Enterprise Onboarding	Growth Onboarding
Duration	30 days	14 days
CSM	Dedicated CSM	Pooled CSM
Kickoff	Internal handoff, kickoff call, success plan sign-off	Kickoff call, success plan
Data Migration	Export source data, field mapping, test import, full import, **Migration verified**	Export, import, **Migration verified**
Configuration	Pipelines, roles and permissions, integrations, SSO	Pipelines, roles
Go-Live	Admin training, user training, go-live checklist, handoff to support	Training, go-live checklist, handoff

- Make the first Go-Live task depend on **Migration verified**.
- Set task durations so the template totals 30 and 14 days.
- These task lists are a demo placeholder. Say so in the video: CS owns the real ones.

Get the template IDs from the template URL or the templates endpoint. Save them as `ROCKETLANE_TEMPLATE_ENTERPRISE` and `ROCKETLANE_TEMPLATE_GROWTH`.

Create a project from a template

Two documented routes. **Verify** the body field names in the API reference before coding.

Route A, one call: `POST /projects` with the template in the body.

```
{
  "projectName": "Acme Corp Onboarding (Enterprise)",
  "customer": { "companyName": "Acme Corp" },
  "owner": { "emailId": "owner@yourdomain.com" },
  "startDate": "2026-11-13",
  "sources": [ { "templateId": 123456, "startDate": "2026-11-13" } ]
}
```

Route B, two calls: `POST /projects`, then `POST /projects/{projectId}/import-template` with `templateId` and `startDate`.

Use Route A if it works. Route B is the fallback.

Duplicate check

List projects and filter by customer company name (**Verify** the available filters). Also store the opportunity ID in a project custom field or in the project name, so the check is reliable.

Overdue escalation (built in the UI, not code)

Settings > Automations > Create. Rocketlane has a native "Task becomes overdue" trigger with a days condition.

Rule	Trigger	Condition	Action
1	Task becomes overdue	Overdue by 1 day	Notify Project Manager
2	Task becomes overdue	Overdue by 4 days	Notify Project Owner

- **Verify** that "Project Manager" exists as a recipient. If only "Project Owner" and named users or roles are offered, add a project-level role or custom field for PM and target that. Mention the choice in the video.
- Attach the rules at the template level so every new project inherits them.
- To demo: create a task with a due date in the past, or show the rule configuration plus the automation run history.

---

5. Gmail + Cloud Pub/Sub

Docs: https://developers.google.com/workspace/gmail/api/guides/push

Gmail does not post to your webhook directly. Gmail publishes to a Pub/Sub topic, and a push subscription forwards to you.

Google Cloud project

1. Create a project at console.cloud.google.com. No billing-heavy services are used. **Verify** whether Pub/Sub asks you to enable billing; usage here sits in the free tier.
2. Enable **Gmail API** and **Cloud Pub/Sub API**.

OAuth

1. **APIs & Services > OAuth consent screen:** External, Testing mode, add your CS-inbox Gmail address as a test user.
2. **Credentials > Create OAuth client ID:** Desktop app. Save the client ID and secret.
3. Scopes: `https://www.googleapis.com/auth/gmail.modify` (read, label, and send replies). Use `gmail.readonly` + `gmail.send` if you prefer narrower scopes.
4. Run a one-time local script with `google-auth-oauthlib` (`InstalledAppFlow.run_local_server`) to get a refresh token. Save it as `GOOGLE_REFRESH_TOKEN` .
5. Testing-mode refresh tokens expire after 7 days. Fine for this assignment.

Pub/Sub

```
gcloud pubsub topics create gmail-deals

gcloud pubsub topics add-iam-policy-binding gmail-deals \
  --member="serviceAccount:gmail-api-push@system.gserviceaccount.com" \
  --role="roles/pubsub.publisher"

gcloud pubsub subscriptions create gmail-deals-push \
  --topic=gmail-deals \
```

```
  --push-endpoint="$PUBLIC_BASE_URL/webhooks/gmail?token=$GMAIL_PUSH_TOKEN" \
  --ack-deadline=10
```

The IAM binding is the step everyone forgets. Without it the watch call fails.

Start the watch

```
gmail.users().watch(userId="me", body={
    "topicName": os.environ["GMAIL_PUBSUB_TOPIC"],
    "labelIds": ["INBOX"],
    "labelFilterBehavior": "INCLUDE",
}).execute()
# returns {"historyId": "...", "expiration": "..."}
```

- Store the returned `historyId` as your starting cursor.
- The watch expires after about 7 days. Re-call it daily in production. For the demo, call `POST /admin/gmail/watch` once before recording.

Handle a push

The push body looks like this:

```
{ "message": { "data": "<base64>", "messageId": "..." }, "subscription": "..." }
```

Decoded `data` is `{"emailAddress": "...", "historyId": 12345}`. It does not contain the email.

Handler steps:

1. Check `?token=` equals `GMAIL_PUSH_TOKEN`. Otherwise 401.
2. Return 200 quickly (inside the 10 s ack deadline). Do the rest in a background task.
3. `users.history.list(userId="me", startHistoryId=<stored cursor>, historyTypes=["messageAdded"], labelId="INBOX")`.
4. For each new message ID: skip if `gmail:{id}` exists, otherwise `users.messages.get(format="full")`.
5. Filter to deal emails (subject contains "Closed Won" or "New deal", or a Gmail label/filter you set up). Ignore everything else, including your own replies.
6. Save the newest `historyId` as the new cursor.
7. Start a run.

If `history.list` returns 404 (cursor too old), fall back to `messages.list` for the last day and reset the cursor.

Reply for clarification

`users.messages.send` with the original `threadId`, and `In-Reply-To` / `References` headers set to the original `Message-ID`, so it lands in the same thread.

Test email

```
Subject: Closed Won: Acme Corp

Hi team,

We just closed Acme Corp.
Customer contact: Dana Lee <dana@acme.example>
Salesforce: https://novacrm.my.salesforce.com/lightning/r/Opportunity/006XXXXXXXXXXXXXX/view
AE: Jai Krishna

Thanks,
Jai
```

Negative version: remove the "Customer contact" line.

---

6. Slack

Docs: https://docs.slack.dev

Workspace

Pick one, in this order:

1. **Free workspace + Pro trial.** Create a workspace, start the Pro trial from the upgrade prompt (no card). Slack Connect becomes available.
2. **Developer Program sandbox.** Join at api.slack.com/developer-program, provision two sandboxes ("NovaCRM", "Acme Corp"). Needs a payment method on file. Slack Connect works only between sandboxes.
3. **Fallback:** one free workspace, add the "customer" as a normal member.

App

1. api.slack.com/apps > **Create New App > From scratch**, in the NovaCRM workspace.
2. **OAuth & Permissions > Bot Token Scopes:**

Scope	For
`channels:manage`	Create channels, set topic, invite members
`chat:write`	Post the welcome message
`conversations.connect:write`	Send Slack Connect invites
`users:read`, `users:read.email`	Look up internal members by email (optional)

3. **Install to Workspace.** Copy the Bot User OAuth Token (`xoxb-...`) to `SLACK_BOT_TOKEN`.

Calls, in order

```
from slack_sdk import WebClient
slack = WebClient(token=os.environ["SLACK_BOT_TOKEN"])

ch = slack.conversations_create(name="onboarding-acme-enterprise")      # bot is auto-joined
cid = ch["channel"]["id"]

slack.conversations_setTopic(channel=cid,
    topic="Acme Corp | Enterprise | 30-day onboarding | CSM: Priya")

slack.conversations_invite(channel=cid, users="U123,U456")              # internal team

slack.conversations_inviteShared(channel=cid, emails=["dana@acme.example"])  # Slack Connect

slack.chat_postMessage(channel=cid, text=welcome_message)
```

Notes:

- Channel names: lowercase, no spaces, max 80 characters. Slugify the customer name.
- `name_taken` on create means a channel exists. Treat it as a duplicate signal: look it up and reuse or escalate.
- `conversations.inviteShared` sends an email invite to the customer. Errors like `not_paid` or `restricted_action` mean Slack Connect is not enabled on the workspace. Catch them, log, and fall back to the internal-only channel with a note in the audit log.
- The customer has to accept the invite. For the demo, accept it from the second workspace or sandbox.

---

7. LLM

1. Create an API key with your provider of choice. Save as `LLM_API_KEY` and set `LLM_MODEL`.
2. Use structured output (JSON schema or tool-call mode) for extraction and transcript interpretation.
3. All calls go through `llm/client.py` so tokens, latency and cost are recorded.
4. Set temperature to 0 for extraction and interpretation.

---

8. PostgreSQL (Docker)

`docker-compose.yml` (core part):

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: postgres
    ports: ["5432:5432"]
    volumes:
      - pgdata:/var/lib/postgresql/data
```

```
      - ./backend/db/init.sql:/docker-entrypoint-initdb.d/init.sql
volumes:
  pgdata:
```

`init.sql`:

```
CREATE DATABASE app;
CREATE DATABASE mig_source;
CREATE DATABASE mig_dest;
```

LangGraph checkpointer:

```
from langgraph.checkpoint.postgres import PostgresSaver

with PostgresSaver.from_conn_string(DATABASE_URL) as checkpointer:
    checkpointer.setup()          # run once, creates checkpoint tables
    graph = builder.compile(checkpointer=checkpointer)
```

- Install: `pip install langgraph langgraph-checkpoint-postgres "psycopg[binary,pool]"`.
- For a long-lived FastAPI app, create a `psycopg_pool.ConnectionPool` at startup and pass it to `PostgresSaver(pool)`. **Verify** the pool connection kwargs (`autocommit=True`, `row_factory=dict_row`) in the LangGraph persistence docs.
- Every `invoke` needs `config={"configurable": {"thread_id": run_id}}`.
- Resume with `graph.invoke(Command(resume=call_result), config)`.

Podman works the same with `podman compose`.

---

9. Salesforce

Not connected. The Opportunity ID is parsed from the link in the email:

```
re.search(r"/Opportunity/([a-zA-Z0-9]{15,18})", body)
```

Production path: Salesforce REST API to confirm the opportunity is Closed Won and to pull the AE's phone number from the User record.

---

10. Pre-recording checklist

- [ ] ngrok running, URL matches Pub/Sub subscription and Retell webhook
- [ ] `POST /admin/gmail/watch` called today
- [ ] Retell balance covers at least 10 minutes
- [ ] Your number is in `ae_directory` in E.164 format
- [ ] Both Rocketlane templates exist, IDs in `.env`

- [ ] Both Rocketlane automations enabled
- [ ] Slack Connect invite works to the customer address
- [ ] No leftover "Acme Corp" project or channel from rehearsals (or use a new customer name)
- [ ] Phone off silent, on the desk, on camera or audible